

```

"""
fase4_boruta.py
=====

FASE 4 (OPCIONAL) - Validacion por importancia multivariada: Boruta / BorutaShap.

Solo se ejecuta si ``usar_boruta = True``.

Que hace Boruta
-----
Boruta es un metodo *all-relevant*: no busca el subconjunto minimo suficiente,
sino TODAS las variables que aportan informacion. El procedimiento es:

1. Por cada variable real se crea una **shadow feature**: una copia con sus
   valores permutados al azar. Por construccion, la sombra conserva la
   distribucion marginal de la variable pero destruye toda relacion con el
   target: es ruido con la misma "forma".
2. Se entrena un Random Forest sobre el conjunto ampliado (reales + sombras).
3. Una variable real "gana el round" si su importancia supera la maxima
   importancia observada entre TODAS las sombras de esa iteracion.
4. Se repite el proceso; los aciertos de cada variable siguen una binomial
   Bin(n_iter, 0.5) bajo la hipotesis nula de irrelevancia. Se contrasta con
   correccion de Bonferroni y cada variable termina como **CONFIRMADA**,
   **RECHAZADA** o **TENTATIVA**.
```

Por que se usa como CONTRASTE y no como reemplazo

```

-----
Boruta mide importancia CONDICIONAL (en presencia de las demas variables),
mientras que IV y Gini miden poder MARGINAL. Cada enfoque ve cosas distintas:

* Una variable con IV alto puede ser rechazada por Boruta si otra la subsume.
* Una variable con IV bajo puede ser confirmada si aporta solo en interaccion
  con otras (algo que una prueba bivariada no puede detectar por diseno).
```

Ademas, la importancia del Random Forest esta sesgada hacia variables de alta cardinalidad, y en datos de panel las observaciones no son independientes (una misma entidad aparece repetida), lo que puede optimizar el resultado. Por eso Boruta informa y matiza, pero no decide por si solo.

Motores disponibles

```

-----
``borutapy``    BorutaPy (libreria externa).
``borutashap``  BorutaShap (importancia via valores SHAP: mas fiel pero mas caro).
``nativo``      Implementacion incluida en este modulo. No depende de librerias
                  externas mas alla de scikit-learn y es la que se usa cuando las
                  otras no estan disponibles o fallan.
``auto``        Intenta borutapy -> borutashap -> nativo, en ese orden.
"""
```

```

from __future__ import annotations

import warnings

import numpy as np
import pandas as pd

from .config import ConfigPipeline
from .logging_utils import obtener_logger

LOGGER = obtener_logger("fase4")

# =====
# Preparacion de la matriz de diseno
# =====
def preparar_matriz(
    df: pd.DataFrame, columnas: list[str], tipos: dict[str, str], cfg: ConfigPipeline
) -> tuple[pd.DataFrame, dict[str, str]]:
    """Convierte las variables seleccionadas en una matriz numerica densa.
```

Tratamientos:

```

* Numericas: los nulos se imputan con la MEDIANA. Se elige la mediana y no
la media por robustez ante colas pesadas; se registra la imputacion.
* Categoricas: codificacion ordinal por frecuencia descendente. No se usa
one-hot para no multiplicar la dimension (y con ella el numero de
sombras, que encarece Boruta cuadraticamente); los arboles pueden
particionar un codigo ordinal sin problema, con la salvedad de que el
orden impuesto es arbitrario.
* Fechas: ordinal (dias desde la epoca).
"""
X = pd.DataFrame(index=df.index)
notas: dict[str, str] = {}

for col in columnas:
    tipo = tipos.get(col, "CATEGORICA")
    serie = df[col]

    if tipo in ("NUMERICA", "BOOLEANA"):
        v = pd.to_numeric(serie, errors="coerce")
        if v.isna().any():
            mediana = v.median()
            mediana = 0.0 if pd.isna(mediana) else float(mediana)
            v = v.fillna(mediana)
            notas[col] = f"numerica; {int(serie.isna().sum())} nulos imputados con la mediana ({mediana:g})"
        else:
            notas[col] = "numerica sin nulos"
        X[col] = v.astype(float)

    elif tipo == "FECHA":
        v = pd.to_datetime(serie, errors="coerce").map(
            lambda d: d.toordinal() if pd.notna(d) else np.nan
        )
        X[col] = v.fillna(v.median() if v.notna().any() else 0).astype(float)
        notas[col] = "fecha convertida a ordinal (dias desde la epoca)"

    else:
        s = serie.astype("object").where(serie.notna(), "__NULOS__").astype(str)
        orden = s.value_counts().index.tolist()
        mapa = {cat: i for i, cat in enumerate(orden)}
        X[col] = s.map(mapa).astype(float)
        notas[col] = f"categorica codificada ordinalmente por frecuencia ({len(orden)} niveles)"

return X, notas

```

```

# =====
# Implementacion nativa de Boruta
# =====
def boruta_nativo(
    X: pd.DataFrame, y: pd.Series, cfg: ConfigPipeline, es_clasificacion: bool = True,
) -> pd.DataFrame:
    """Algoritmo Boruta implementado sobre scikit-learn.

```

Sin dependencias externas: garantiza que la fase 4 pueda ejecutarse siempre que el nucleo del proyecto este instalado.

Returns

```

pd.DataFrame
    Columnas: ``columna``, ``aciertos``, ``n_iteraciones``,
    ``importancia_media``, ``importancia_sombra_max_media``,
    ``p_valor``, ``estado``.
"""

```

```

from scipy.stats import binomtest
from sklearn.ensemble import RandomForestClassifier, RandomForestRegressor

```

```

rng = np.random.default_rng(cfg.semilla)
columnas = list(X.columns)
n_vars = len(columnas)

```

```

aciertos = pd.Series(0, index=columnas, dtype=int)
imp_acum = pd.Series(0.0, index=columnas, dtype=float)

```

```

sombra_max_acum: list[float] = []
n_iter_efectivas = 0

Modelo = RandomForestClassifier if es_clasificacion else RandomForestRegressor
kwargs = dict(
    n_estimators=cfg.boruta_n_estimadores,
    max_depth=cfg.boruta_profundidad_max,
    n_jobs=cfg.n_jobs,
    random_state=cfg.semilla,
)
if es_clasificacion:
    # Balanceo por clase: con eventos escasos, sin esto el bosque aprende
    # a predecir siempre la clase mayoritaria y ninguna variable destaca.
    kwargs["class_weight"] = "balanced_subsample"

Xv = X.to_numpy(dtype=float)
yv = y.to_numpy()

LOGGER.info("Boruta nativo: %d variables, %d filas, hasta %d iteraciones.",
            n_vars, len(Xv), cfg.boruta_max_iter)

for it in range(1, cfg.boruta_max_iter + 1):
    # 1. Sombras: permutacion independiente de cada columna real.
    sombras = np.column_stack([rng.permutation(Xv[:, j]) for j in range(n_vars)])
    Xamp = np.hstack([Xv, sombras])

    # 2. Ajuste del bosque sobre reales + sombras.
    modelo = Modelo(**kwargs, random_state=cfg.semilla + it)
    with warnings.catch_warnings():
        warnings.simplefilter("ignore")
        modelo.fit(Xamp, yv)

    imp = np.asarray(modelo.feature_importances_, dtype=float)
    imp_reales, imp_sombras = imp[:n_vars], imp[n_vars:]

    # 3. Umbral del round: la mejor sombra observada.
    umbral = float(np.max(imp_sombras)) if len(imp_sombras) else 0.0
    sombra_max_acum.append(umbral)
    aciertos += (imp_reales > umbral).astype(int)
    imp_acum += imp_reales
    n_iter_efectivas = it

    if it % 10 == 0:
        LOGGER.info("    iteracion %d/%d | umbral sombra=%.6f | confirmadas provisionales=%d",
                    it, cfg.boruta_max_iter, umbral,
                    int((aciertos >= it * 0.75).sum()))

# 4. Test binomial con correccion de Bonferroni.
# H0: la variable gana el round con probabilidad 0.5 (es ruido).
# Se corrige por el numero de variables porque se realizan n_vars
# contrastes simultaneos; sin la correccion, con 50 variables se
# esperarían ~2.5 falsos positivos solo por azar.
alpha = cfg.boruta_alpha / max(n_vars, 1)
filas = []
for col in columns:
    h = int(aciertos[col])
    p_conf = binomtest(h, n_iter_efectivas, 0.5, alternative="greater").pvalue
    p_rech = binomtest(h, n_iter_efectivas, 0.5, alternative="less").pvalue

    if p_conf < alpha:
        estado = "Confirmed"
    elif p_rech < alpha:
        estado = "Rejected"
    else:
        estado = "Tentative"

filas.append({
    "columna": col,
    "aciertos": h,
    "n_iteraciones": n_iter_efectivas,
    "tasa_aciertos": h / n_iter_efectivas if n_iter_efectivas else np.nan,

```

```

        "importancia_media": float(imp_acum[col] / n_iter_efectivas) if n_iter_efectivas else np.nan,
        "importancia_sombra_max_media": float(np.mean(sombra_max_acum)) if sombra_max_acum else np.nan,
        "p_valor_confirmacion": float(p_conf),
        "p_valor_rechazo": float(p_rech),
        "alpha_bonferroni": alpha,
        "estado": estado,
    })

return pd.DataFrame(filas)

# =====
# Motores externos
# =====
def _boruta_libreria(X: pd.DataFrame, y: pd.Series, cfg: ConfigPipeline,
                    es_clasificacion: bool) -> pd.DataFrame | None:
    """Intenta ejecutar BorutaPy. Devuelve ``None`` si no es viable."""
    try:
        # BorutaPy usa alias de numpy retirados en NumPy >= 1.24 (np.float,
        # np.int, np.bool). Se restauran para que la libreria pueda importarse
        # sin modificar su codigo fuente. `np.object` se omite a proposito:
        # NumPy lo intercepta con un FutureWarning propio y BorutaPy no lo usa.
        with warnings.catch_warnings():
            warnings.simplefilter("ignore")
            for alias, tipo in (("float", float), ("int", int), ("bool", bool)):
                if not hasattr(np, alias):
                    setattr(np, alias, tipo)

        from boruta import BorutaPy
        from sklearn.ensemble import RandomForestClassifier, RandomForestRegressor

        Modelo = RandomForestClassifier if es_clasificacion else RandomForestRegressor
        kwargs = dict(n_estimators=cfg.boruta_n_estimadores, max_depth=cfg.boruta_profundidad_max,
                      n_jobs=cfg.n_jobs, random_state=cfg.semilla)
        if es_clasificacion:
            kwargs["class_weight"] = "balanced_subsample"

        selector = BorutaPy(
            Modelo(**kwargs), n_estimators="auto", max_iter=cfg.boruta_max_iter,
            alpha=cfg.boruta_alpha, random_state=cfg.semilla, verbose=0,
        )
        with warnings.catch_warnings():
            warnings.simplefilter("ignore")
            selector.fit(X.to_numpy(dtype=float), y.to_numpy())

        estados = []
        for i, col in enumerate(X.columns):
            if selector.support_[i]:
                estado = "Confirmed"
            elif selector.support_weak_[i]:
                estado = "Tentative"
            else:
                estado = "Rejected"
            estados.append({
                "columna": col,
                "ranking_boruta": int(selector.ranking_[i]),
                "estado": estado,
                "n_iteraciones": int(getattr(selector, "n_iter_", cfg.boruta_max_iter) or cfg.boruta_max_iter),
            })
        return pd.DataFrame(estados)

    except ImportError as exc:
        LOGGER.warning("BorutaPy no esta disponible (%s).", exc)
        return None
    except Exception as exc: # noqa: BLE001
        LOGGER.warning("BorutaPy fallo en ejecucion (%s). Se usara otro motor.", exc)
        return None

def _borutashap_libreria(X: pd.DataFrame, y: pd.Series, cfg: ConfigPipeline,
                        es_clasificacion: bool) -> pd.DataFrame | None:

```

```

"""Intenta ejecutar BorutaShap. Devuelve ``None`` si no es viable."""
try:
    from BorutaShap import BorutaShap
    from sklearn.ensemble import RandomForestClassifier, RandomForestRegressor

    Modelo = RandomForestClassifier if es_clasificacion else RandomForestRegressor
    modelo = Modelo(n_estimators=cfg.boruta_n_estimadores, max_depth=cfg.boruta_profundidad_max,
                    n_jobs=cfg.n_jobs, random_state=cfg.semilla)

    selector = BorutaShap(model=modelo, importance_measure="shap",
                          classification=es_clasificacion, percentile=100, pvalue=cfg.boruta_alpha)
    with warnings.catch_warnings():
        warnings.simplefilter("ignore")
        selector.fit(X=X, y=y, n_trials=min(cfg.boruta_max_iter, 50),
                    random_state=cfg.semilla, verbose=False)

    confirmadas = set(getattr(selector, "accepted", []) or [])
    tentativas = set(getattr(selector, "tentative", []) or [])
    filas = []
    for col in X.columns:
        if col in confirmadas:
            estado = "Confirmed"
        elif col in tentativas:
            estado = "Tentative"
        else:
            estado = "Rejected"
        filas.append({"columna": col, "estado": estado})
    return pd.DataFrame(filas)

except ImportError as exc:
    LOGGER.warning("BorutaShap no esta disponible (%s).", exc)
    return None
except Exception as exc: # noqa: BLE001
    LOGGER.warning("BorutaShap fallo en ejecucion (%s).", exc)
    return None

# =====
# Orquestacion de la fase
# =====
def ejecutar(
    df: pd.DataFrame,
    cfg: ConfigPipeline,
    tipos: dict[str, str],
    variables: list[str],
    tipo_target: str,
    biv: pd.DataFrame,
    multi: pd.DataFrame,
) -> tuple[pd.DataFrame, dict]:
    """Ejecuta la fase 4 y contrasta sus resultados con las fases previas.

    Returns
    -----
    (tabla_boruta, metadatos)
    """
    meta = {"ejecutada": False, "motor": "", "n_confirmadas": 0, "n_rechazadas": 0,
            "n_tentativas": 0, "observacion": "", "n_filas_usadas": 0}

    if not cfg.usar_boruta:
        meta["observacion"] = "Fase omitida: el parametro usar_boruta es False."
        LOGGER.info("FASE 4 - OMITIDA (usar_boruta=False).")
        return pd.DataFrame(), meta

    LOGGER.info("=" * 78)
    LOGGER.info("FASE 4 - BORUTA / BORUTASHAP (VALIDACION COMPLEMENTARIA)")
    LOGGER.info("=" * 78)

    if len(variables) < 2:
        meta["observacion"] = (
            f"Se requieren al menos 2 variables y llegaron {len(variables)}. "
            "Boruta necesita comparar importancias entre variables y sus sombras."

```

```

)
LOGGER.warning(meta["observacion"])
return pd.DataFrame(), meta

# --- Preparacion de datos -----
y_full = pd.to_numeric(df[cfg.columna_target], errors="coerce")
mascara = y_full.notna()
dfx = df.loc[mascara]
y = y_full.loc[mascara]

# Submuestreo estratificado por periodo: preserva la estructura temporal
# del panel en lugar de tomar una muestra aleatoria simple que podria
# sobrerrepresentar unos periodos frente a otros.
if cfg.boruta_max_filas and len(dfx) > cfg.boruta_max_filas:
    frac = cfg.boruta_max_filas / len(dfx)
    idx = (
        dfx.groupby(cfg.columna_tiempo, observed=True, group_keys=False)
        .apply(lambda g: g.sample(frac=frac, random_state=cfg.semilla), include_groups=False)
        .index
    )
    dfx, y = dfx.loc[idx], y.loc[idx]
    LOGGER.info("Submuestreo estratificado por '%s': %d filas para Boruta.",
                cfg.columna_tiempo, len(dfx))

meta["n_filas_usadas"] = int(len(dfx))
X, notas = preparar_matriz(dfx, variables, tipos, cfg)
es_clasificacion = tipo_target in ("BINARIO", "MULTICLASE")
y_modelo = y.astype(int) if es_clasificacion else y.astype(float)

# --- Seleccion de motor con degradacion controlada -----
resultado, motor = None, ""
orden = {
    "auto": ["borutapy", "borutashap", "nativo"],
    "borutapy": ["borutapy", "nativo"],
    "borutashap": ["borutashap", "nativo"],
    "nativo": ["nativo"],
}[cfg.motor_boruta]

for candidato in orden:
    LOGGER.info("Intentando motor '%s'...", candidato)
    if candidato == "borutapy":
        resultado = _boruta_libreria(X, y_modelo, cfg, es_clasificacion)
    elif candidato == "borutashap":
        resultado = _borutashap_libreria(X, y_modelo, cfg, es_clasificacion)
    else:
        resultado = boruta_nativo(X, y_modelo, cfg, es_clasificacion)
    if resultado is not None and not resultado.empty:
        motor = candidato
        break

if resultado is None or resultado.empty:
    meta["observacion"] = "Ningun motor de Boruta pudo ejecutarse; la fase 4 no aporta evidencia."
    LOGGER.error(meta["observacion"])
    return pd.DataFrame(), meta

LOGGER.info("Motor utilizado: %s", motor)

# --- Consolidacion y contraste con las fases previas -----
resultado = resultado.rename(columns={"estado": "boruta_status"})
resultado["motor_boruta"] = motor
resultado["borutashap_status"] = (
    resultado["boruta_status"] if motor == "borutashap" else "NO_APLICA"
)
resultado["flg_confirmada"] = (resultado["boruta_status"] == "Confirmed").astype(int)
resultado["flg_rechazada"] = (resultado["boruta_status"] == "Rejected").astype(int)
resultado["flg_tentativa"] = (resultado["boruta_status"] == "Tentative").astype(int)
resultado["tratamiento_aplicado"] = resultado["columna"].map(notas).fillna("")

if not biv.empty():
    ref = biv.set_index("columna")[["iv", "gini", "score_compuesto", "ranking_score"]]
    resultado = resultado.merge(ref, left_on="columna", right_index=True, how="left")

```

```

if not multi.empty:
    sel_final = set(multi.loc[multi["flg_seleccion_final"] == 1, "columna"])
    resultado["flg_seleccion_fases_1_3"] = resultado["columna"].isin(sel_final).astype(int)
else:
    resultado["flg_seleccion_fases_1_3"] = 1

def _concordancia(f: pd.Series) -> str:
    """Compara el veredicto de Boruta con el de las fases clasicas."""
    clasica, boruta = f["flg_seleccion_fases_1_3"] == 1, f["flg_confirmada"] == 1
    if clasica and boruta:
        return "COINCIDEN_SELECCIONAN"
    if not clasica and not boruta:
        return "COINCIDEN_DESCARTAN"
    if clasica and not boruta:
        return "SOLO_CLASICAS (poder marginal, no aporta condicionalmente)"
    return "SOLO_BORUTA (aporta en interaccion; revisar reincorporacion)"

resultado["concordancia_con_fases_previas"] = resultado.apply(_concordancia, axis=1)
resultado = resultado.sort_values(
    ["flg_confirmada", "score_compuesto"], ascending=[False, False], na_position="last"
).reset_index(drop=True)

meta.update(
    ejecutada=True, motor=motor,
    n_confirmadas=int(resultado["flg_confirmada"].sum()),
    n_rechazadas=int(resultado["flg_rechazada"].sum()),
    n_tentativas=int(resultado["flg_tentativa"].sum()),
    observacion=(
        f"Motor '{motor}' sobre {len(X)} filas y {len(variables)} variables. "
        "Resultado usado como CONTRASTE: no modifica automaticamente la seleccion final."
    ),
)

LOGGER.info("Confirmadas: %d | Tentativas: %d | Rechazadas: %d",
            meta["n_confirmadas"], meta["n_tentativas"], meta["n_rechazadas"])
conteo = resultado["concordancia_con_fases_previas"].value_counts().to_dict()
LOGGER.info("Concordancia con las fases 1-3: %s", conteo)

return resultado, meta

```